

FitMate Frontend — React 마이그레이션 사전 분석

분석 대상: frontend/ 디렉토리 (vanilla JS + HTML + CSS). backend/ 는 분석 범위에서 제외.
목적: React로 마이그레이션하기 위한 페이지/기능/상태/의존관계 인벤토리와 전환 시 참고사항 정리.

1. 페이지/기능 인벤토리

폴더	진입점	담당 기능
Page/Login/	Login.html	로그인 폼, 이메일/비밀번호 유효성 검사(Login.js:17-47), 로그인 성공 시 accessToken 저장 후 서버가 내려준 링크로 이동(Login.js:71,74)
Page/Signup/	Signup.html	회원가입 폼(이메일/비번/비번확인/닉네임/프로필사진), blur 시 유효성 검사, 409 중복 응답 처리(Signup.js:197-216)
Page/Board/	board.html	게시글 목록, IntersectionObserver 기반 무한스크롤(board.js:30-59), 사이드바/드롭다운
Page/Post_detail/	post_detail.html	게시글 상세, 댓글 CRUD, 좋아요, 신고, 게시글 삭제(모달), 수정 페이지 이동
Page/Post_write/	post_write.html	게시글 작성 폼(제목/내용/이미지)
Page/Post_edit/	post_edit.html	게시글 수정 폼, 진입 시 기존 값 프리필(post_edit.js:61-73)
Page/Profile_edit/	profile_edit.html	회원정보 조회/수정, 회원탈퇴(모달), 로그아웃
Page/Password_edit/	password_edit.html	비밀번호 변경
Page/Modal/	없음(modal.css 만 존재)	모달 공용 스타일시트, JS/HTML 엔트리 없음 — post_detail.html:9, profile_edit.html:9 에서 <link> 로만 참조

페이지 이동 흐름 (근거: 전체 href/onclick/ location.href grep)

```
Login.html —(회원가입 링크, Login.html:61)→ Signup/signup.html ※대소문자 불일치, 5절 참고
Login.html —(로그인 성공, Login.js:74, 서버가 내려준 response.data.link)→ (보통 Board.html)

Signup.html —(뒤로가기, Signup.html:14 / 로그인하려가기, Signup.html:71)→ Login/Login.html
Signup.html —(가입 성공, Signup.js:195, result.data.link)→ (보통 Login.html)

Board.html —(게시글 카드 클릭, board.js:187-189)→ Post_detail/post_detail.html?postId=N
Board.html —(게시글 작성 버튼, board.html:75)→ Post_write/post_write.html
Board.html —(드롭다운, board.html:47-48)→ Profile_edit/profile_edit.html, Password_edit/password_edit.html

Post_detail.html —(목록으로, post_detail.html:72)→ Board/board.html
Post_detail.html —(수정 버튼, post_detail.js:139)→ Post_edit/post_edit.html?postId=N
Post_detail.html —(삭제 성공, post_detail.js:150 / 신고 성공, post_detail.js:348)→ Board/board.html

Post_write.html —(뒤로가기, post_write.html:14)→ Board/board.html
Post_write.html —(작성 성공, post_write.js:82, response.data.link)→ (보통 Post_detail.html)

Post_edit.html —(뒤로가기, post_edit.html:14)→ Board/board.html
Post_edit.html —(수정 성공, post_edit.js:101)→ Post_detail/post_detail.html?postId=N

Profile_edit.html —(로그아웃, profile_edit.js:61)→ Login/login.html ※대소문자 불일치
Profile_edit.html —(회원탈퇴 성공, profile_edit.js:127, response.data.link)→ (보통 Login.html)
```

모든 성공 후 이동이 클라이언트에 하드코딩된 라우트가 아니라 서버 응답의 data.link 값을 그대로 신뢰하는 구조입니다 (Login.js:74, Signup.js:195, Post_write.js:82, Profile_edit.js:127). React Router 설계 시 이 계약을 유지할지, 클라이언트 라우트 상수로 대체

할지 결정이 필요합니다.

2. 주요 함수 및 역할 (파일별)

API/request.js

함수	라인	역할
<code>needsAuth(path)</code>	8-10	경로가 인증 헤더 필요 있는지 판단
<code>request(path, method, data)</code>	18-35	공용 API 호출 함수, 인증 헤더 부착
<code>handleResponse(response)</code>	37-80	상태코드별 분기 처리, 에러에 <code>.status</code> / <code>.field</code> 부착

Page/Login/Login.js

- 이벤트 핸들러: `emailInput` `input`(17-31), `passwordInput` `input`(33-47), `loginButton` `click`(63-79)
- API 호출: `login(login_user)` (59-61)
- 렌더링: 없음 (`helperText` `textContent`만 갱신, DOM 생성 없음)

Page/Signup/Signup.js

- 이벤트 핸들러: `profileInput` `change`(40-65), `emailInput` / `passwordInput` / `confirmPasswordInput` / `nicknameInput` `blur`(70-166), `signupButton` `click`(184-217)
- 검증/UI: `setProfileInvalid()` (27-34), `activeSignupButton()` (168-176)
- API 호출: `signUp(signUp_user)` (180-182)

Page/Board/board.js

- API 호출: `getListPost()` (61-83, `cursor` 페이지네이션)
- 렌더링(DOM 생성): `createStatItem(iconKey, count)` (115-120), `renderPostList(posts)` (122-195, `createElement` + `appendChild` 방식, `innerHTML` 은 아이콘 SVG 삽입에만 사용, 118)
- 유틸: `formatCount(count)` (86-91), `formatDateTime(dateInput)` (94-105)
- 이벤트 핸들러: `profileMenuBtn` `click`(19-21), `intersectionObserver` 콜백(30-57, 무한스크롤), `sidebarCollapseBtn` / `sidebarExpandBtn` `click`(199-207)

Page/Post_detail/post_detail.js

- API 호출: `getDetailPost()` (102-104), `deletePost()` (143-145), `createComment()` (157-159), `editComment()` (217-219), `deleteComment()` (222-224), `likePost()` (309-311), `unlikePost()` (314-316), `reportPost()` (339-341) — 8개 API 함수, 파일 중 가장 많음
- 렌더링: `createCommentElement(comment)` (162-214, DOM 생성 방식), 초기 렌더링은 `DOMContentLoaded` (106-136)에서 `textContent` 직접 대입
- 유틸: `formatCount` (74-79), `formatDateTime` (83-94) — `board.js`와 완전히 동일한 코드 중복
- 이벤트 핸들러: 좋아요(318-335), 신고(344-352), 댓글 등록/수정 토글(278-305), 댓글 목록 이벤트 위임(`commentList.addEventListener`, 240-265), 각종 모달 열고닫기(48-57, 227-230, 260-264)

Page/Post_write/post_write.js

- 이벤트 핸들러: `titleInput` / `postContentInput` `input`(23-31), `postImageInput` `change`(33-42)
- 검증: `activeWriteCompleteButton()` (44-56)
- API 호출: `writePost(Post_data)` (59-61)
- 미사용 변수: `userId` (16, 3절 참고)

Page/Post_edit/post_edit.js

- post_write.js와 거의 동일 구조(대부분 로직 복제): `activeEditCompleteButton()` (44-55), `defaultEditPage()` (61-64, 진입 시 기존값 조회), `editPost(Post_data)` (77-79)
- 미사용 변수: `userId` (16)

Page/Profile_edit/profile_edit.js

- API 호출: `getUser()` (75-88), `withdrawUser()` (108-122) — 주의: `request.js` 를 안 쓰고 직접 `fetch` 사용 (4절 참고), `updateUser(update_User)` (134-136, 이걸 request 사용)
- 렌더링: `showToast()` (138-144)
- 이벤트 핸들러: 프로필 사진 미리보기(35-57), 로그아웃(59-62), 탈퇴모달(66-72), 정보수정 제출(146-219)

Page/Password_edit/password_edit.js

- 이벤트 핸들러: 비밀번호/확인 blur 검증(23-74)
- API 호출: `passwordEdit(update_password)` (86-88) — 버그: `userId` 가 이 파일에서 전혀 정의되지 않은 채 참조됨(87)

3. 상태 관리 방식 분석

전역 변수로 관리되는 상태

모두 모듈 스코프 `let`, 프레임워크 없이 클로저로 공유됩니다.

- `Page/Board/board.js:24` `cursorId`, `:27` `isLoading` — 무한스크롤 페이지네이션 상태
- `Page/Post_detail/post_detail.js:39-40` `isEditing`, `isLiked`, `:233-237` `currentEditCommentId` / `currentEditCommentBody` / `currentDeleteCommentId` / `currentDeleteItem` — 댓글 수정/삭제 대상 추적, 좋아요 여부
- 각 폼 페이지의 `isValidEmail` / `isValidPassword` / `isValidNickname` / `isValidProfile` 등 (`Login.js:11-12`, `Signup.js:20-24`, `Profile_edit.js:27-28`, `Password_edit.js:15-16`) — 필드별 유효성 플래그, 버튼 활성화 조건으로만 사용

localStorage (근거: `grep localStorage` 전체 결과)

`accessToken` 만 저장됨. `Login.js:71` 저장 → `API/request.js:23` 매 요청마다 읽어 헤더 부착 → `Profile_edit.js:60` 로그아웃 시 삭제. `Profile_edit.js:79,113` 에서도 직접 읽는데, 이는 `request.js` 를 거치지 않는 우회 호출(4절 참고).

sessionStorage

`userId` 키를 읽기만 함(`Post_write.js:16`, `Post_edit.js:16`, `Profile_edit.js:109`) — 코드 전체에 `sessionStorage.setItem` 이 한 번도 없어 항상 null. `Post_write.js` / `Post_edit.js`에서는 읽어온 뒤 어디에도 사용하지 않는 죽은 코드. `Password_edit.js:87` 은 아예 이 패턴조차 없이 미정의 `userId` 를 참조(런타임 에러 가능성, 5절 참고).

DOM 자체에 상태를 저장하는 패턴

- `classList` 토글: `active` (드롭다운/모달/버튼 활성화 공통), `liked` (`post_detail.js:124,322,327`), `collapsed` (사이드바, `board.js:200,205`), `error` (헬퍼텍스트), `show` (토스트)
- `hidden` 속성 토글: 로딩/빈목록/에러 문구(`board.js:38-54`), 사이드바 접힘 상단바(`board.js:201,206`)
- `data-*` 속성: 댓글 ID를 버튼에 저장 — `post_detail.js:193` `editBtn.dataset.commentId`, `:198` `deleteBtn.dataset.commentId`, 클릭 시 `:246,260` 에서 다시 읽음. 사실상 DOM이 댓글-ID 매핑 상태 저장소 역할

상태 전파 방식

이벤트 발행/구독(pub-sub) 패턴 없음. 모두 클로저 변수 직접 참조 또는 이벤트 위임(`post_detail.js:240` `commentList.addEventListener` 로 자식 버튼 클릭까지 위임)으로 처리. 페이지 간 상태 전파는 전역 스토어가 아니라 **URL 쿼리 파라미터**(? `postId=`, `post_detail.js:97-98`, `post_edit.js:57-58`)와 **페이지 재진입 시 서버 재조회**(`DOMContentLoaded` 시점 API 호출)로만 이루어짐.

4. 파일 의존 관계

```
API/request.js
├─ import: Page/Login/Login.js:1
├─ import: Page/Signup/Signup.js:1
├─ import: Page/Board/board.js:1
├─ import: Page/Post_detail/post_detail.js:1
├─ import: Page/Post_write/post_write.js:1
├─ import: Page/Post_edit/post_edit.js:1
├─ import: Page/Profile_edit/profile_edit.js:1 (단, 일부 호출은 우회함 - 아래 참고)
└─ import: Page/Password_edit/password_edit.js:1
```

- **API/request.js** 는 8개 페이지 JS 전부에서 import되는 유일한 공용 모듈입니다. 페이지 JS끼리 서로 import하는 경우는 없습니다(각 파일 완전 독립).
- **예외/우회**: Profile_edit/profile_edit.js 는 request 를 import하고도 getUser() (76-88)와 withdrawUser() (110-115)에서 request() 를 쓰지 않고 fetch 를 직접 호출하며 BASE_URL 도 http://localhost:8080 으로 재하드코딩합니다. 이 두 호출은 request.js 의 401/403/404/409/500 공통 에러 처리(handleResponse)를 전혀 타지 않습니다.
- CSS 의존: Page/Modal/modal.css 가 post_detail.html:9, profile_edit.html:9 에서 <link> 로 공유됩니다. 모달 마크업 자체는 이 두 페이지(+ Profile_edit 의 탈퇴모달)에만 존재합니다.
- 그 외 정적 리소스 의존은 전부 외부 CDN(pretendard.css , 각 HTML 7-9번 줄)뿐, 로컬 공용 컴포넌트/모듈은 없습니다.

5. React 전환 관점 참고사항

그대로 옮기기 쉬운 부분

- API/request.js 전체 — fetch 래퍼 로직은 프레임워크 무관, axios 인스턴스나 커스텀 훅(useApi)으로 거의 그대로 이식 가능
- 각 품의 정규식 기반 유효성 검사 로직(이메일/비밀번호 정규식은 Login.js:19,35, Signup.js:72,89,118, Password_edit.js:26,58 에 걸쳐 100% 동일 문자열로 4번 중복) — 순수 함수라 validators.ts 유틸로 추출하면 그대로 재사용
- formatCount / formatDate (board.js:86-105 = post_detail.js:74-94 , 완전 동일 중복) — 공용 유틸로 추출

구조 변경이 필요한 부분

- **공용 헤더/사이드바**: profileMenuBtn + dropdownMenu +로그아웃 마크업이 Board, Post_detail, Post_write, Post_edit, Profile_edit, Password_edit 6개 HTML에 거의 동일하게 중복되어 있으나, **로그아웃 버튼 이벤트는 Profile_edit/profile_edit.js:59-62 에만 등록**되어 있어 나머지 5개 페이지에서는 로그아웃 버튼을 눌러도 아무 반응이 없는 실제 버그입니다. 컴포넌트로 통합할 때 반드시 공용 로직으로 고쳐야 합니다.
- **모달**: 게시글 삭제(post_detail.html:142-153)/댓글 삭제(:156-167)/회원탈퇴(profile_edit.html:81-92) 모달이 마크업·열고닫기 로직(classList.add('active') / modal-open) 패턴이 동일 — 공용 ConfirmModal 컴포넌트 후보
- **토스트**: showToast() 가 Profile_edit.js:138-144 와 Password_edit.js:90-96 에 완전히 동일하게 복제됨 — 공용 Toast 컴포넌트/훅 후보
- **게시글 작성/수정 폼**: Post_write 와 Post_edit 는 제목/내용/이미지 필드 마크업과 로직이 90% 이상 동일 — mode="create"|"edit" prop을 받는 공용 PostForm 컴포넌트로 통합 가능
- **무한스크롤**: IntersectionObserver 로직(board.js:30-59)은 useInfiniteScroll 커스텀 훅으로 추출하기 좋은 후보
- **댓글 리스트**: 이벤트 위임(post_detail.js:240-265)으로 구현된 수정/삭제는 React에서는 댓글 아이템 컴포넌트 + onEdit / onDelete 콜백 props로 자연스럽게 전환 가능
- **인증 상태**: localStorage.accessToken 을 여러 파일에서 직접 읽고 쓰는 구조 → Context/전역 스토어(Zustand 등)로 통합 필요. 특히 Profile_edit.js 의 직접 fetch 우회 부분은 통합 시 없어져야 함

마이그레이션 전 반드시 확인해야 할 버그/이상 동작

위치	내용
Password_edit.js:87	<code>userId</code> 변수가 파일 어디에도 정의되지 않은 채 템플릿 리터럴에서 참조됨 → 비밀번호 변경 API 호출 시 <code>ReferenceError</code> 로 항상 실패할 가능성이 높음(런타임에서 실제 동작하는지 검증 필요)
Board.js / Post_detail.js / Post_write.js / Post_edit.js / Password_edit.js 의 <code>logoutBtn</code>	HTML에는 버튼이 있지만(<code>board.html:49</code> 등) JS 이벤트 리스너가 없음 — <code>Profile_edit.js:59</code> 에서만 등록됨
Login.html:61	<code>href="../Signup/signup.html"</code> (소문자) vs 실제 파일명 <code>Signup.html</code> (대문자) — 대소문자 무관 파일시스템(macOS/Windows)에서는 동작하지만 대소문자 구분 배포 환경에서는 깨짐
Profile_edit.js:61	<code>'../Login/login.html'</code> (소문자) vs 실제 파일명 <code>Login.html</code> (대문자) — 위와 동일한 대소문자 불일치
Post_write.js:16 , Post_edit.js:16	<code>sessionStorage.getItem("userId")</code> 로 읽어온 <code>userId</code> 를 선언만 하고 실제로는 어디에도 사용하지 않는 죽은 코드
Profile_edit.js:76-88,108-122	<code>request.js</code> 를 우회한 직접 <code>fetch</code> 호출 — 공통 에러 처리(401 alert 등)가 적용되지 않음